

RESULTS

Exam Results

901 / 1000

PASSED

54 of 60 correct (90%) · Pass mark: 720

Domain Breakdown

D1

Agentic Architecture & Orchestration

14/14 (100%) +270pts

D2

Tool Design & MCP Integration

8/11 (73%) +131pts

D3

Claude Code Configuration & Workflows

10/12 (83%) +167pts

D4

Prompt Engineering & Structured Output

11/12 (92%) +183pts

D5

Context Management & Reliability

11/11 (100%) +150pts

HOW SCORING WORKS

Each domain's percentage correct is multiplied by its weight and scaled to 1000 points. For example, scoring 80% in Agentic Architecture (27% weight) contributes $0.80 \times 0.27 \times 1000 = 216$ points. The pass mark is 720/1000 (72%). If a domain has no questions in the selected scenarios, its weight is redistributed proportionally among assessed domains.

The real exam reports a scaled score from 100 to 1,000 plus your percent-correct by domain, so treat the percentages here as a readiness gauge rather than a prediction of your scaled score.

HIDE DETAIL

TAKE ANOTHER EXAM

Question-by-Question Review

ALL 60

CORRECT 54

INCORRECT 6

SKIPPED 0

Q01 · D5 5.5 HUMAN-REVIEW-CALIBRATION / AGGREGATE-METRICS-TRAP q-5-5-004 · Structured Data Extraction Pipeline

CORRECT

A structured data extraction system processes invoices, purchase orders, and contracts. The monitoring dashboard shows 97% overall extraction accuracy and the team considers the system production-ready. A detailed audit reveals: invoices 99.5% accuracy (80% of volume), purchase orders 98% accuracy (15% of volume), contracts 72% accuracy (5% of volume). What does this reveal and what action is required?

A

The system is performing well. 72% on contracts is acceptable since contracts represent only 5% of volume and barely affect the overall metric

Acceptability depends on the business impact, not volume percentage. Contract extraction errors may have severe financial or legal consequences despite low volume. A 28% error rate on any document type is significant.

C

The training data needs rebalancing so that contracts represent a larger proportion, which will naturally improve contract accuracy

LLMs used for extraction are not retrained on production data this way. The issue is monitoring visibility and operational response, not model training data distribution.

D

The 97% threshold should be raised to 99% overall to force improvement across all document types

Raising the aggregate threshold does not address per-type disparities. Even at 99% overall, contract accuracy could remain low if invoice volume continues to dominate the metric. The solution is per-type metrics, not a higher aggregate bar.

B

The aggregate metric is masking a severe per-type disparity. Give contracts mandatory review and report per-type metrics.

This is a textbook case of aggregate metrics hiding per-type problems. 97% overall is misleading because high-volume invoice accuracy (99.5%) overwhelms the poor contract performance (72%). Per-type metrics would have surfaced this immediately. Contracts need different treatment until the extraction quality is acceptable.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Aggregate metrics trap)

Q02 · D4 4.3 STRUCTURED-OUTPUT / TOOL-CHOICE-FORCING q-4-3-002 · Structured Data Extraction Pipeline INCORRECT

Your legal document extraction pipeline uses `tool_use` with `tool_choice` set to `'auto'`. The pipeline processes contracts and sometimes receives plain text analysis instead of the structured JSON extraction you need. The team suggests switching to prompt-based JSON with explicit formatting instructions. What is the correct approach?

B Keep `tool_use` but switch `tool_choice` from `'auto'` to `'any'` to guarantee the model always returns a structured tool call

While `'any'` guarantees a tool call, it lets the model choose which tool. If you have multiple tools defined, the model might call the wrong one. For a single extraction tool, this works, but the most precise solution is to force the specific extraction tool. (your answer)

D Add stronger instructions in the system prompt telling the model to always use the extraction tool and never respond with plain text

Instructions alone cannot guarantee tool use when `tool_choice` is `'auto'`. The API-level `tool_choice` parameter is the correct mechanism for enforcing structured output, not prompt instructions.

C Keep `tool_use` and set `tool_choice` to force the specific extraction tool by name, guaranteeing structured output on every request.

Forcing a specific tool with `tool_choice {type: 'tool', name: 'extract_contract'}` guarantees the model calls that exact tool every time, eliminating both text-only responses and wrong-tool selection. This is the most reliable approach for guaranteed structured output. (correct answer)

A Switch to prompt-based JSON as suggested, since the model clearly prefers text responses for these documents

Prompt-based JSON is less reliable than `tool_use` and can produce malformed output. The issue is `tool_choice` configuration, not the `tool_use` mechanism itself.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (`tool_choice` forcing)

Anthropic: Tool Use Documentation ↗

Q03 · D2 2.5 BUILT-IN-TOOLS / EDIT-RECOVERY q-2-5-007 · Developer Productivity Tools INCORRECT

A developer asks Claude Code to modify a function in a large file. The Edit tool fails with a `'non-unique match'` error because the target text appears in multiple locations within the file. What is the correct recovery approach?

B Use Read to load the full file contents, then use Write to output the complete modified file with the change applied.

Rewriting the whole file with Write is wasteful and unsafe — it loses the surgical-edit guarantees of Edit and risks dropping content. This is a last resort when no unique context exists, not the designed recovery path.

C Expand the Edit search string to include more surrounding context until it becomes unique in the file.

The Edit tool's documented recovery path: 'Either provide a larger string with more surrounding context to make it unique, or use `replace_all` to change every instance of `old_string`.' Expanding context preserves Edit's surgical safety. If every occurrence should be changed identically, `replace_all` is the alternative. (correct answer)

D Split the file into smaller files so that each occurrence appears in only one file, then use Edit on each.

Restructuring files to accommodate tool limitations is disproportionate effort. The Edit tool's own recovery path (more context or `replace_all`) handles non-unique matches without changing the codebase.

A Switch to Bash with sed to perform the replacement using line numbers instead of text matching.

Bash with sed bypasses the built-in Edit tool's safety guarantees and is fragile when line numbers shift. The Edit tool documents the correct first recovery step (expand context or use `replace_all`). (your answer)

WHERE THIS COMES FROM

Lesson 2.5: Built-in Tools (Edit non-unique match recovery)

Claude Code: Built-in Tools ›

Q04 · D2 2.4 MCP-SERVER-INTEGRATION / DESCRIPTIONS q-2-4-006 · Enterprise Data Platform with Federated Queries CORRECT

An MCP server exposes a `query_database` tool for Snowflake. Agents ignore it and run SQL via the built-in Bash tool against the Snowflake CLI, even though the MCP tool returns structured results with column types and pagination that Bash does not. What is the most likely cause and fix?

B Enhance the sparse description to spell out the tool's structured output and pagination advantages over Bash.

When MCP tool descriptions are sparse, agents default to the familiar Bash tool. Enhancing the description to explain the structured results, column types, and pagination gives the agent enough to prefer the MCP tool for database queries.

D Add a system prompt instruction telling the agent to always use `query_database` instead of Bash for SQL queries.

Whilst a system prompt instruction could work as a short-term fix, it is brittle and does not scale. The root cause is the sparse description. Enhancing the description is the sustainable fix that works across all agents and contexts.

A The MCP server is not properly connected. Restart the MCP server and verify the connection with a test query.

If the server were disconnected, the tool would not appear in the available tools list at all. The agent is choosing not to use it, which is a description problem, not a connectivity problem.

C Disable the Bash tool entirely so the agent is forced to use the MCP tool for all operations.

Disabling Bash removes a critical general-purpose tool needed for many operations beyond database queries. The fix should make the MCP tool more attractive for its specific use case, not remove other tools.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Enhancing MCP descriptions)

Q05 · D5 5.1 CONTEXT-WINDOW-MANAGEMENT / TOOL-RESULT-TRIMMING q-5-1-005 · Enterprise Data Platform with Federated Queries CORRECT

An analytics agent queries Snowflake and receives 40+ columns per row, only 5 of which are relevant to the user's question about quarterly revenue. The agent appends the full result set to context. After three such queries the window is nearly full and follow-up questions fail. What is the most effective fix?

C Store all query results in an external database and have the agent retrieve specific values on demand instead of keeping results in context.

External storage adds infrastructure complexity without addressing the core issue. The agent still needs some result data in context to reason about it. Trimming irrelevant fields is simpler and directly reduces context consumption.

A Upgrade to a model with a larger context window so that full result sets can be accommodated across more queries.

A larger context window postpones the problem but does not solve it. Each full result set still wastes tokens on 35+ irrelevant columns, and the context will eventually fill regardless of size.

B Trim tool results to only the relevant columns before appending them to the conversation context.

Tool result trimming removes irrelevant fields before they enter the context window. Keeping only the 5 relevant columns from each 40+ column result set dramatically reduces token consumption, allowing the agent to handle many more queries within the same context budget.

D Limit the number of rows returned by each query to reduce the total data volume in context.

Row limits reduce data volume but may exclude relevant rows. The problem is column-level verbosity (35+ irrelevant columns per row), not row count. Trimming columns preserves all relevant rows whilst eliminating irrelevant fields.

WHERE THIS COMES FROM

Lesson 5.1: Context Window Management (Tool result trimming)

Lesson 5.1: Context Window Management (Upstream optimisation)

Q06 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / SCOPED-TOOLS q-1-3-017 · Multi-Agent Research System CORRECT

A web search agent has 9 tools: 'web_search', 'url_fetch', 'html_parse', 'pdf_extract', 'image_ocr', 'translate', 'summarise', 'keyword_extract', 'sentiment_analysis'. In testing it frequently calls 'summarise' and 'sentiment_analysis' when it should only fetch raw data. How should the architect fix this?

D Merge the web search and synthesis agents into a single agent since they share some tools

Merging agents violates the principle of specialised subagents. The solution is to separate tools by role, not to combine agents. Merging would create an even larger tool set on a single agent.

A Add system prompt instructions telling the web search agent to only use data fetching tools and ignore analysis tools

Prompt instructions are probabilistic and the agent is already misusing tools. The architectural fix is to remove the tools it should not have, providing deterministic enforcement through scoped tool access.

B Reduce the web search agent to the 4-5 data-fetching tools and move the analysis tools to the specialist agents.

The recommended maximum is 4-5 tools per agent. The web search agent has 9 tools, causing it to use tools outside its intended role. Reducing to data-fetching tools enforces scoped access. The analysis tools (summarise, sentiment_analysis, keyword_extract) belong with the synthesis or analysis agents.

C Keep all 9 tools but implement PreToolUse hooks that block the web search agent from calling summarise and sentiment_analysis

Using hooks to block tool access is an over-engineered workaround for what should be a configuration fix. The proper solution is to remove the tools from allowedTools, not to provide them and then block their use.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Scoped tool access (4-5 tools))

Anthropic: Claude Agent SDK Overview ↗

Q07 · D3 3.1 CLAUDE-MD-HIERARCHY / ENFORCEMENT q-3-1-007 CORRECT

Project-level '.claude/CLAUDE.md' says 'use 4-space indentation matching the existing codebase.' A senior architect has 'use 2-space indentation' in their user-level '~/.claude/CLAUDE.md'. In recent sessions the architect's code has come back in 2 spaces and broken the build. The team needs a guarantee that 4-space indentation is applied on every save. What should they do?

C Ask the architect to delete their user-level '~/.claude/CLAUDE.md' so there is no conflict to resolve

This treats a personal config file as a team problem and doesn't scale — every new teammate would need to police their own home directory, and any future contradiction (from a different file, an '@import', or a '.claude/rules/' entry) would resurface the same fragility. The fix is to remove reliance on guidance-style config for a rule that must be enforced, not to remove the conflicting file.

A Add a PostToolUse hook that runs the team's formatter after every Write/Edit, so 4-space indentation is enforced regardless of what Claude generates

Anthropic's memory docs are explicit that CLAUDE.md is delivered as a user message with 'no guarantee of strict compliance,' and that 'if two rules contradict each other, Claude may pick one arbitrarily.' The docs themselves point at hooks for this case: hooks 'execute as shell commands at fixed lifecycle events and apply regardless of what Claude decides to do.' A PostToolUse hook running the formatter is the only option here that gives a hard guarantee.

B Leave the rule in project-level '.claude/CLAUDE.md' — the more specific scope wins on conflicts, so the project rule will override the architect's user-level preference

This is the popular paraphrase, but it's not what Anthropic's docs say. The docs describe a load order (broadest scope to most specific, so project instructions appear in context after user instructions) but explicitly state files are 'concatenated into context rather than overriding each other' and conflicts 'may [be] pick[ed] arbitrarily.' The team has already seen that assumption fail in production.

D Move the 4-space rule into a 'CLAUDE.local.md' at the project root so it is appended last and reads after the user-level file

Load order does append 'CLAUDE.local.md' after 'CLAUDE.md' within a directory, but the docs are careful to call this 'load order,' not precedence, and warn that conflicts may resolve arbitrarily. 'CLAUDE.local.md' is also gitignored, so a team standard cannot live there.

WHERE THIS COMES FROM

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (Loading order and conflict handling)

Q08 · D3 3.1 CLAUDE-MD-HIERARCHY / HIERARCHY q-3-1-005 · Developer Productivity Tools CORRECT

A consultancy works across 12 client projects simultaneously. Each developer has personal preferences (editor keybindings, alias shortcuts) and the consultancy has firm-wide coding standards. Each client project has its own specific conventions. Some client projects have subsystems with additional specialised rules. What is the correct configuration architecture?

D User ~/.claude/CLAUDE.md for everything; use @ path imports to pull in project-specific files from each repository

User-level CLAUDE.md loads for every session across all projects. Importing 12 client project configurations into the user-level file would load all client conventions for every project, wasting tokens and causing conflicting instructions.

A User ~/.claude/CLAUDE.md for personal preferences; project .claude/CLAUDE.md for firm-wide standards and client conventions; directory-level CLAUDE.md or .claude/rules/ with paths for subsystem rules

Personal preferences are per-developer, so they live at user level, which is not shared via version control. Firm-wide standards and client conventions must reach everyone who works on a project, so they belong at the project level, which is committed to the repository and distributed on clone; firm-wide standards can be kept in a shared file and pulled into each project's CLAUDE.md with an @import to stay modular. Putting shared standards at the user level is the classic hierarchy bug, where a colleague who clones a project would not receive them. Subsystem rules belong at the directory level or in path-scoped .claude/rules/.

C A single CLAUDE.md at the root of each project containing all four levels of configuration, with clear section headings

Combining all configuration levels into a single file defeats the purpose of the hierarchy. Personal preferences would be version-controlled and shared with the team, firm-wide standards would need duplication across 12 projects, and the file would be bloated with subsystem rules that only apply to specific directories.

B User ~/.claude/CLAUDE.md for personal preferences and firm-wide standards; project .claude/CLAUDE.md for client-specific conventions; directory-level CLAUDE.md or .claude/rules/ with paths for subsystem-specific rules

User-level ~/.claude/CLAUDE.md applies only to the individual developer and is not shared via version control. Placing firm-wide standards there means a colleague who clones a client project does not receive them, which is the classic configuration-hierarchy bug. Shared standards belong at the project level.

WHERE THIS COMES FROM

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (Three-level hierarchy)

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (.claude/rules/)

Q09 · D4 4.4 VALIDATION-RETRY / RETRY-WITH-ERROR q-4-4-007 · Structured Data Extraction Pipeline CORRECT

Your extraction pipeline has a validation step that checks extracted financial data. When validation fails, the system retries by sending just the original document with the same prompt. Retry success rates are below 10%. The most common failure is the model placing the 'net amount' value in the 'gross amount' field and vice versa. How should the retry be restructured?

A Increase retries from 1 to 5, since more attempts at the same prompt will eventually produce the correct field mapping by chance

If the model consistently maps fields incorrectly with the same prompt, more retries without feedback will produce the same incorrect mapping. The model needs to see its specific error.

C Add a post-processing rule that automatically swaps net and gross amounts when validation detects the pattern

Hard-coded swap rules are brittle and mask the underlying extraction error. They also fail if the issue manifests differently in other documents. Error feedback teaches the model to extract correctly.

D Switch to a different model for the retry, since the original model has a persistent field-mapping bias

The issue is the absence of error context in the retry, not a model-specific bias. Any model retrying without seeing the specific validation error will have a low success rate.

- B** Send the original document, the failed extraction, and the specific validation error naming the swapped fields.

Retry-with-error-feedback is dramatically more effective than naive retries. The model seeing its specific mistake (net and gross amounts swapped) and the validation error allows targeted self-correction rather than repeating the same misinterpretation.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry with error feedback)

Lesson 4.4: Validation, Retry, and Feedback Loops (Self-correction flow)

Q10 · D5 5.4 CODEBASE-EXPLORATION / COMPACT q-5-4-006 · Enterprise Data Platform with Federated Queries **CORRECT**

A developer has been using the data platform agent in a single extended session for two hours, running complex federated queries, debugging SQL syntax, and iterating on report formatting. The agent now produces noticeably worse SQL and occasionally references table schemas from earlier queries that have since been corrected. What is the most effective intervention?

- A** Use `/compact`, optionally with focus instructions, to summarise the conversation and free context budget.

Extended sessions cause context degradation as stale information (outdated schemas, abandoned approaches) competes with current context. `/compact` summarises the conversation history into a compressed form, freeing context budget. The optional `[instructions]` argument lets you bias the summary (for example, 'preserve the corrected schema for the orders table'). It is the documented mitigation for context degradation in long sessions, and the least disruptive option of the four.

- D** Add a system prompt instruction telling the agent to ignore all schema information from more than 30 minutes ago.

The agent has no reliable concept of message timestamps within the context window. Time-based instructions are unenforceable. The structural fix is to compress or remove stale context entries.

- B** Start a completely new session and re-establish all context from scratch.

A fresh session discards all accumulated context, including valuable recent work. `/compact` summarises the existing conversation while preserving the gist of recent work, which is less disruptive than starting over.

- C** Increase the model's temperature parameter to improve creativity and reduce fixation on stale patterns.

Temperature affects output randomness, not context quality. The problem is stale information in the context window polluting the agent's reasoning, not a lack of creative variation.

WHERE THIS COMES FROM

Lesson 5.4: Codebase Exploration and Context Degradation (`/compact`)

Q11 · D2 2.5 BUILT-IN-TOOLS / EDIT-RECOVERY q-2-5-002 · Developer Productivity Tools **INCORRECT**

A developer is using Claude Code to update a configuration value in a file. They use the Edit tool, but it fails because the text they are trying to match appears in three places in the file. What is the correct next step?

- A** Switch to using Bash with `sed` to perform the replacement, as it handles multiple matches natively.

`Sed` bypasses the built-in tool designed for this task and may introduce errors with special characters. The exam expects candidates to use the correct built-in tool sequence. (your answer)

- C** Fall back to Read to load the full file, then Write the complete modified file.

Read + Write is the fallback when Edit cannot be made to work at all. The correct first step is to provide more surrounding context to make the Edit match unique. Read + Write should only be used after that approach fails.

- B** Use Grep to find which of the three occurrences is the correct one, then provide more surrounding context to Edit to make the match unique.

When Edit fails due to non-unique matches, the first recovery step is to include more surrounding context so the match becomes unique. Grep helps identify the right occurrence and its context. This keeps you within the Edit workflow. (correct answer)

D Use Glob to find all files containing the configuration value, then edit each one individually.

Glob searches file paths, not file contents. It cannot find files containing a specific value. Additionally, the problem is within a single file, not across multiple files.

WHERE THIS COMES FROM

Lesson 2.5: Built-in Tools (Edit tool)

Claude Code: Built-in Tools ↗

Q12 · D3 3.3 PATH-SPECIFIC-RULES / GLOB-PATTERNS q-3-3-011

INCORRECT

Test-file conventions must apply to *.test.tsx files spread across many directories of a codebase. Why is a .claude/rules/ file with a paths glob the right mechanism? (Select 2)

B The rule is appended to every session's context, so the conventions are never missed.

Always-on loading is what path-specific rules avoid; a rule that loads everywhere is just CLAUDE.md content. (your answer)

A One glob such as **/*.test.tsx captures the files by type wherever they live in the tree.

Glob patterns apply conventions by file type regardless of directory, which per-directory files cannot do cleanly.

C The rule loads only when Claude edits a matching file, keeping irrelevant context and token usage out of other sessions.

Conditional activation on path match is the core benefit of path-scoped rules. (correct answer)

D A CLAUDE.md file in each directory would achieve the same effect with less duplication.

Scattered test files would need a copy of the convention in every directory, which is exactly the duplication glob-scoped rules remove.

WHERE THIS COMES FROM

Lesson 3.3: Path-Specific Rules (Path-specific rules)

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (.claude/rules/ directory)

Q13 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / STRUCTURED-METADATA q-1-3-001 · Multi-Agent Research System

CORRECT

A synthesis agent produces a report with several claims that have no source attribution. The web search and document analysis subagents are working correctly and returning well-sourced results. What is the most likely root cause?

D The synthesis agent needs access to the web search tool directly

Giving the synthesis agent web search tools violates the principle of scoped tool access and does not address the context passing failure.

A The synthesis agent's system prompt does not instruct it to include citations, so it drops source attribution even when given well-sourced input.

Even with a citation instruction, the synthesis agent cannot attribute claims to sources that were never passed to it. The lesson is explicit: do not blame the synthesis prompt; the root cause is the coordinator dropping structured metadata.

C The web search subagent needs to return results in a different format

The web search subagent is working correctly. The issue is how the coordinator passes its results to the synthesis agent.

B The coordinator passes content to the synthesis agent stripped of structured metadata like source URLs and document names.

Context passing must include structured data that separates content from metadata. Without source URLs and document names, the synthesis agent has no attribution information to include.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Structured metadata format)

Lesson 1.3: Subagent Invocation and Context Passing (Context passing)

Q14 · D1 1.6 TASK-DECOMPOSITION / ATTENTION-DILUTION q-1-6-001 · Multi-Agent Research System CORRECT

A code review agent processes 14 files and produces detailed feedback for the first 5 files but misses obvious bugs in files 10-14. It also flags a pattern as problematic in one file while approving identical code in another. What is the root cause and solution?

B Split the review into per-file local analysis passes plus a separate cross-file integration pass

Multi-pass architecture solves attention dilution. Per-file passes ensure consistent depth for each file. The cross-file integration pass catches data flow issues and ensures consistent pattern evaluation.

C Add a stronger system prompt emphasising the importance of reviewing all files equally

Prompt improvements do not solve attention dilution. The fundamental issue is processing too many items in a single pass, which is an architectural problem requiring a structural solution.

A The model's context window is too small — upgrade to a model with a larger context window

Context window size is not the issue. Attention dilution occurs because processing too many items in a single pass produces inconsistent depth, regardless of window size.

D Reduce the number of files reviewed per run to 5 and process in batches

Batching is closer to the right idea but misses the cross-file integration pass. Without it, cross-file data flow issues and pattern consistency are not addressed.

WHERE THIS COMES FROM

Lesson 1.6: Task Decomposition Strategies (Attention dilution)

Lesson 1.6: Task Decomposition Strategies (Sequential pipelines)

Q15 · D5 5.3 ERROR-PROPAGATION / SILENT-SUPPRESSION q-5-3-005 · Multi-Agent Research System CORRECT

A multi-agent system has subagents for web search, academic database, and industry reports. The academic database subagent hits a 403 Forbidden on a specific journal, catches the error, and returns an empty result set marked as successful. The coordinator produces a final report with no academic sources. What is the critical failure in this design?

D The entire workflow should have been terminated when the academic database subagent encountered the 403 error.

Workflow termination on a single subagent failure wastes partial results from the web search and industry reports subagents that may have completed successfully. The coordinator should proceed with available results and annotate the coverage gap.

B The subagent silently suppressed the access failure by returning empty results as success, preventing the coordinator from attempting recovery or noting the gap in coverage.

Silent suppression — returning empty results marked as success — is the worst error propagation anti-pattern. The coordinator believes the academic search succeeded and found nothing, so it produces a report without academic sources and without noting the coverage gap. The subagent should have returned structured error context with the failure type, what was attempted, and potential alternatives.

A The coordinator should validate that all subagents return non-empty results before producing the final report.

Requiring non-empty results from all subagents would cause the workflow to fail when a subagent legitimately finds no matching data. The issue is that the error was suppressed, not that the coordinator failed to validate result volume.

C The subagent should have retried the request with exponential backoff, as 403 errors are often transient.

403 Forbidden is a permission error, not a transient error. Retrying will not resolve it — the subagent lacks the required access credentials. The correct response is to propagate the error with its type so the coordinator knows the failure is not retryable.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Silent suppression anti-pattern)

Lesson 5.3: Error Propagation in Multi-Agent Systems (Access failure vs empty)

Q16 · D1 1.6 TASK-DECOMPOSITION / COORDINATOR q-1-6-007 · Multi-Agent Research System CORRECT

A multi-agent research system must process a customer's request that involves three sequential stages: data collection, analysis, and report generation. Each stage depends on the output of the previous one. Which orchestration pattern is most appropriate?

B Pipeline orchestration — pass the output of each stage as input to the next in a defined sequence

Pipeline orchestration is the correct pattern for sequential dependencies. Each stage completes before the next begins, with the output of one stage serving as the input to the next. This matches the data collection → analysis → report generation dependency chain.

C Dynamic adaptive decomposition — let the coordinator decide the order at runtime based on query complexity

Dynamic adaptive decomposition is for open-ended investigation tasks where the next step depends on discoveries. Here, the three stages and their dependencies are known in advance, making a fixed pipeline the more predictable and efficient choice.

A Parallel orchestration — run all three subagents simultaneously to minimise latency

Parallel execution requires independent tasks. Here, analysis depends on data collection results and report generation depends on analysis results. Running them simultaneously would mean the analysis agent has no data to analyse.

D Hub-and-spoke with all three agents reporting independently to the coordinator

Independent reporting ignores the sequential dependencies. The analysis agent cannot produce meaningful results without data collection output. The pipeline must enforce the execution order.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q17 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ERROR-CATEGORIES q-2-2-006 · Multi-Agent Research System CORRECT

A 'search_papers' MCP tool has three failure patterns: (1) the upstream academic API returns HTTP 503 during peak hours, (2) users request papers from a restricted journal the system has no licence for, and (3) the agent submits a malformed DOI like 'doi-1234' that fails the input regex. The team wants structured error metadata so agents can handle each case differently. Which 'errorCategory' and 'isRetryable' combination is correct for all three?

A All three should be errorCategory: 'transient', isRetryable: true, since they all prevent the tool from completing its task.

Lumping every failure into 'transient' is the canonical anti-pattern. The restricted journal case is a business error (licence policy); the agent must escalate, not retry. The malformed DOI is validation; the agent must repair the input format first.

B HTTP 503: transient/retryable; restricted journal: business/not retryable; malformed DOI: validation/retryable.

HTTP 503 is a temporary upstream outage (transient, will likely resolve on retry). Restricted journal is a policy limitation (business, will never resolve on retry — escalate or offer an alternative source). Malformed DOI is validation; the agent must repair the input and then retry.

C HTTP 503: transient/retryable; restricted journal: transient/retryable; malformed DOI: transient/retryable.

The restricted journal error is not transient — the system lacks a licence, and no amount of retrying will grant access. A malformed DOI is validation, not transient; the agent must repair the input rather than blindly retry the same value.

D HTTP 503: validation/retryable; restricted journal: business/not retryable; malformed DOI: business/not retryable.

HTTP 503 is transient (a temporary upstream outage), not validation — nothing is wrong with the request input. A malformed DOI is validation (fix input and retry), not business — business errors represent policy or rule violations, not input format problems.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Four error categories)

Q18 · D5 5.3 ERROR-PROPAGATION / STRUCTURED-ERROR-CONTEXT q-5-3-001 · Multi-Agent Research System CORRECT

A web search subagent in a multi-agent research system times out while researching a complex topic. You need to design how this failure information flows back to the coordinator. Which approach best enables intelligent recovery?

D Propagate the timeout exception to a top-level handler that terminates the entire research workflow

Workflow termination wastes partial results from other subagents that may have completed successfully.

A Return structured error context including failure type, attempted query, partial results, and potential alternative approaches

This gives the coordinator everything it needs to decide: retry with modified query, try an alternative approach, or proceed with partial results.

C Catch the timeout and return an empty result set marked as successful

Silent suppression prevents any recovery. The coordinator believes the search succeeded and found nothing, so it will not attempt alternatives.

B Implement automatic retry with exponential backoff, returning a generic 'search unavailable' status only after all retries are exhausted

The generic status hides valuable context from the coordinator, preventing informed recovery decisions.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Structured error context)

Q19 · D1 1.6 TASK-DECOMPOSITION / ADAPTIVE-DECOMPOSITION q-1-6-002 · Multi-Agent Research System CORRECT

A consulting firm needs an agent to add a comprehensive test suite to a large legacy codebase. The codebase has no existing tests, unclear dependencies between modules, and the team does not know which areas are most critical. Which task decomposition strategy is most appropriate?

A A fixed sequential pipeline that reviews each file in alphabetical order and generates tests for each one

Fixed sequential pipelines are best for predictable, structured tasks. Adding tests to a legacy codebase with unclear dependencies is an open-ended investigation that requires adaptive exploration to discover which areas are most critical and how dependencies affect test design.

D A multi-pass architecture with per-file analysis and a cross-file integration pass, exactly like a standard code review pipeline.

Multi-pass architecture solves attention dilution for review tasks, but test generation in a legacy codebase needs adaptive exploration: discovering critical modules and dependencies and prioritising coverage. The decomposition must adapt to findings, not follow a fixed per-file pattern.

C Dynamic adaptive decomposition: map the codebase, find the high-impact areas, and adapt the plan as dependencies emerge.

Adding tests to a legacy codebase with unknown dependencies is an open-ended investigation task. Dynamic adaptive decomposition generates subtasks based on what is discovered at each step — first mapping the structure, then identifying critical areas, then adapting the test plan as dependency relationships emerge.

B A single-pass analysis that processes the entire codebase at once and generates a complete test plan

A single-pass analysis of a large codebase would suffer from attention dilution, producing inconsistent coverage. Additionally, a single pass cannot adapt to discoveries about dependencies and critical areas that emerge during exploration.

WHERE THIS COMES FROM

Lesson 1.6: Task Decomposition Strategies (Dynamic adaptive decomposition)

Lesson 1.6: Task Decomposition Strategies (Pattern selection)

Q20 · D2 2.4 MCP-SERVER-INTEGRATION / BUILD-VS-USE q-2-4-001 · Developer Productivity Tools CORRECT

A team needs to integrate with Jira for issue tracking in their Claude Code workflow. A developer proposes building a custom MCP server. What is the correct first step?

D Add the Jira integration to ~/.claude.json so each developer can configure it independently.

A team-wide integration should be in project-level .mcp.json so it is version-controlled and shared. User-level ~/.claude.json is for personal/experimental servers.

A Build a custom MCP server with the exact API endpoints the team needs.

Building custom is premature. Community MCP servers for Jira already exist and cover standard use cases. Custom builds should be reserved for team-specific workflows.

B Evaluate existing community MCP servers for Jira and only build custom if they cannot handle team-specific workflows.

Community servers should always be the first choice for standard integrations. They are maintained, tested, and cover common use cases. Custom builds are justified only when community servers cannot handle team-specific requirements.

C Use the Jira REST API directly from Bash commands instead of MCP.

Direct API calls bypass the MCP tool interface, losing the benefits of tool descriptions, structured responses, and agent-native integration.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Build-vs-use)

Claude Code: MCP Server Configuration ↗

Q21 · D3 3.3 PATH-SPECIFIC-RULES / GLOB-PATTERNS q-3-3-009 · Developer Productivity Tools CORRECT

A developer working on a React Native project notices that Claude Code applies web-specific React conventions (e.g., using div elements) when editing mobile components in the src/mobile/ directory, despite having mobile-specific conventions documented. The root CLAUDE.md contains general React standards, and there is no directory-level or rules-based configuration for mobile. What is the best fix?

C Replace the root CLAUDE.md web React conventions with mobile React Native conventions

Replacing web conventions with mobile conventions would break guidance for the web portion of the codebase. Both web and mobile conventions are needed, but scoped to their respective file paths.

A Add mobile-specific conventions to the root CLAUDE.md alongside the existing web React standards

Adding mobile conventions to the root CLAUDE.md would load them for every session, including when editing web components, API handlers, or any other file. This wastes tokens and could cause confusion when both web and mobile conventions are present simultaneously.

D Create a CLAUDE.md file in src/mobile/ and remove all React standards from the root CLAUDE.md

A directory-level CLAUDE.md in src/mobile/ would work for that directory, but removing React standards from the root CLAUDE.md would leave web components without guidance. The correct approach preserves root-level standards and adds path-scoped mobile rules.

B Create a .claude/rules/react-native.md file with paths: ["src/mobile/**/*"] containing the mobile-specific conventions

A rules file with a path restriction to src/mobile/ ensures mobile conventions load only when editing mobile components. This keeps the root CLAUDE.md focused on universal standards while providing targeted, token-efficient mobile guidance.

WHERE THIS COMES FROM

Lesson 3.3: Path-Specific Rules (Rule file examples)

Lesson 3.3: Path-Specific Rules (Path-specific rules)

Q22 · D5 5.6 INFORMATION-PROVENANCE / CONFLICT-HANDLING q-5-6-005 · Multi-Agent Research System CORRECT

A synthesis agent receives findings from three subagents: solar investment grew 15% (Bloomberg, January 2024), solar investment grew 22% (IEA, March 2024), and cost-per-watt data. The synthesis agent currently picks the IEA figure because it is more recent. What is the correct approach?

A Always use the most authoritative source (IEA as an international agency) and discard the Bloomberg figure.

Discarding a credible source based on perceived authority silently removes valid data. Both sources are credible, and the difference may reflect different methodologies, scopes, or time periods.

B Average the two values (18.5%) and cite both sources to present a balanced view.

Averaging conflicting statistics is mathematically misleading. The two figures may measure different scopes (e.g., different regions or investment types), making an average meaningless. The difference needs to be preserved, not merged.

D Flag the conflicting figures and pause report generation until a human researcher resolves the discrepancy.

Pausing for human resolution is unnecessary when the difference can be explained by temporal or methodological factors. Proper annotation with provenance resolves the ambiguity without blocking the workflow.

C Present both figures with source attribution and dates, noting what may explain the difference.

Preserving both data points with provenance — source, date, and potential explanations for the difference — lets the consumer interpret the data correctly. Different publication dates and methodologies explain the discrepancy without requiring the synthesis agent to judge which is 'right'.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflict handling)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Temporal awareness)

Q23 · D1 1.1 AGENTIC-LOOPS / LIFECYCLE q-1-1-001

CORRECT

A developer's customer support agent sometimes terminates prematurely because they check if `response.content[0].type == 'text'` to determine completion. The agent stops even when Claude has more tools to call. What is the bug and how should they fix it?

B Check the `stop_reason` field instead of content type: continue on 'tool_use', terminate on 'end_turn'.

The `stop_reason` field is the deterministic, authoritative signal for loop control. Claude can return text alongside `tool_use` blocks, so checking content type is unreliable.

A Add an iteration cap of 15 loops and treat reaching the cap as the signal that the agent has finished its work.

Arbitrary caps do not address the root cause, and using the cap as a completion signal is itself an anti-pattern. The agent exits early because it misreads the response type; the fix is to inspect `stop_reason`, not to cap iterations.

D Set `tool_choice` to 'any' so Claude always calls a tool instead of returning text

This forces tool use even when the agent is genuinely finished, creating an infinite loop rather than fixing the detection logic.

C Parse the assistant's text for phrases like 'I have completed' before terminating

Natural language parsing is ambiguous and unreliable. The `stop_reason` field already provides an unambiguous signal.

WHERE THIS COMES FROM

Lesson 1.1: Agentic Loops (Agentic loop lifecycle)

Anthropic: Messages API Reference ↗

Q24 · D2 2.3 TOOL-DISTRIBUTION-CHOICE / ROLE-SCOPING q-2-3-007 · Enterprise Data Platform with Federated Queries

CORRECT

An enterprise data platform agent has 22 tools available, including query tools for Snowflake, PostgreSQL, and third-party APIs, plus data transformation, visualisation, export, and administrative tools. The team notices the agent frequently selects the wrong query tool — choosing the Snowflake tool for data that lives in PostgreSQL. What is the most effective architectural fix?

D Consolidate all query tools into a single `universal_query` tool that internally routes to the correct data source based on the query syntax.

A universal query tool hides the data source distinction from the agent. When the agent needs to reason about which data source contains the relevant data, it loses the ability to make that decision explicitly.

A Improve the descriptions of all 22 tools to include explicit boundaries and data source mappings.

Task Statement 2.1 does teach descriptions as the first fix for misrouting, which is why this option is tempting, but that rule assumes the agent has a workable number of tools and simply cannot tell two of them apart. Here there are 22, well past the 4-5 range where selection stays reliable. Decision complexity degrades selection regardless of how well each description is written, so rewriting all 22 treats the symptom and leaves the cause.

B Split the agent into role-specific agents — a query agent with 4-5 data source tools, a transformation agent, and an export agent — each scoped to its specialisation.

Tool overload at 22 tools degrades selection reliability. Splitting into role-specific agents with 4-5 tools each reduces decision complexity and scopes each agent to its specialisation. The query agent can focus on selecting the right data source without being distracted by transformation or export tools.

C Add a pre-processing step that analyses the user query to determine which data source is needed, then use tool_choice forced selection to call the correct query tool.

Forced tool selection based on a pre-processing step is over-engineered and brittle. It bypasses the LLM's ability to reason about which tool to use and breaks when queries span multiple data sources.

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (Role-specific scoping)

Lesson 2.3: Tool Distribution and Tool Choice (Tool overload)

Q25 · D3 3.2 SLASH-COMMANDS-SKILLS / SKILLS-FRONTMATTER q-3-2-013 · Developer Productivity Tools CORRECT

A team has a /deploy-check skill that verifies deployment readiness. It needs to: (1) read configuration files across the repo, (2) run bash commands to check service health, (3) produce a lengthy multi-page report, and (4) never modify any source code. The skill should be available to the entire team. Which SKILL.md configuration satisfies all four requirements?

A Place in .claude/skills/ with allowed-tools: ["Read", "Grep", "Glob", "Bash"] and context: fork

The skill is in .claude/skills/ for team access (requirement 4 via version control). allowed-tools includes Read, Grep, and Glob for reading configuration files (requirement 1), Bash for running health check commands (requirement 2), and excludes Write and Edit to prevent source code modification (requirement 4). context: fork isolates the lengthy report from the main conversation (requirement 3).

C Place in ~/.claude/skills/ with allowed-tools: ["Read", "Grep", "Glob", "Bash", "Write"] and context: fork

Two problems: ~/.claude/skills/ is user-scoped, not shared with the team. Including Write in allowed-tools violates the requirement that the skill must never modify source code.

D Place in .claude/skills/ with allowed-tools: ["Read", "Grep", "Glob", "Bash"]

This configuration is missing context: fork. Without it, the lengthy multi-page deployment report would clutter the main conversation context, violating requirement 3.

B Place in .claude/skills/ with allowed-tools: ["Read", "Grep", "Glob"] and context: fork

This configuration excludes Bash, which is needed for running service health check commands (requirement 2). Without Bash access, the skill cannot verify that services are running and responsive.

WHERE THIS COMES FROM

Lesson 3.2: Custom Slash Commands and Skills (Skills frontmatter)

Claude Code: Skills ↗

Q26 · D3 3.3 PATH-SPECIFIC-RULES / PATHS-FRONTMATTER q-3-3-008 CORRECT

A large TypeScript monorepo has a root CLAUDE.md with universal coding standards and a .claude/rules/ directory with topic-specific rule files. A developer notices that conventions from testing.md in .claude/rules/ are loading even when editing API handler files, consuming unnecessary tokens. The testing.md file has no YAML frontmatter. What should the developer do to fix this?

D Use an @./claude/rules/testing.md line in the root CLAUDE.md so the file only loads when the import is reached

@ path imports load eagerly when the CLAUDE.md is loaded — the imported file's content is inlined at that moment, regardless of which file the developer is editing. They are a source-organisation mechanism, not a conditional-loading mechanism. Path-scoped frontmatter in .claude/rules/ is the only way to make rule files load only for matching paths.

A Move testing.md out of .claude/rules/ and into a directory-level CLAUDE.md inside the test folder

Test files are co-located with source files throughout 200+ packages in this monorepo. A directory-level CLAUDE.md in a single test folder would not cover test files in other directories, and creating one in every directory would be unmaintainable.

C Add YAML frontmatter with a paths field of test-file globs to testing.md so it loads only for test files

Without YAML frontmatter, rule files in .claude/rules/ load for all sessions. Adding a paths field with glob patterns restricts loading to sessions where the developer is editing matching files. This is the correct approach for token-efficient, conditional convention loading.

B Add the testing conventions to the root CLAUDE.md instead, since rules files cannot be path-scoped

Rules files absolutely support path scoping via YAML frontmatter with a paths field. Moving conventions to root CLAUDE.md would make the token efficiency problem worse, not better, because root CLAUDE.md loads for every session.

WHERE THIS COMES FROM

Lesson 3.3: Path-Specific Rules (Path-specific rules)

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (.claude/rules/ directory)

Q27 · D3 3.5 ITERATIVE-REFINEMENT / INTERVIEW-PATTERN q-3-5-004 · Developer Productivity Tools CORRECT

A developer is new to a healthcare compliance domain and needs to build an audit logging system. They understand the technical implementation options but are unsure about regulatory requirements that might affect the design. Which Claude Code workflow technique is most appropriate?

D Start with direct execution using a simple implementation and iterate based on test failures

Iterating from test failures works when the requirements are known and tests can be written upfront. Healthcare compliance requirements that the developer does not know about cannot be captured in tests. Missing a regulatory requirement is not something that shows up as a test failure.

C Use the interview pattern so Claude asks clarifying questions about compliance requirements, data retention policies, and access control needs before proposing a design

The interview pattern is specifically designed for unfamiliar domains where the developer might miss important considerations. By having Claude ask probing questions about regulatory requirements, the developer surfaces constraints they might not have considered, leading to a more compliant design.

B Use plan mode to have Claude explore the codebase and propose multiple audit logging architectures

Plan mode is for evaluating implementation strategies when the requirements are understood. The developer's gap is in domain requirements (regulatory compliance), not in technical approach selection.

A Provide concrete input/output examples of the desired audit log format and let Claude implement it

Concrete examples fix inconsistent interpretation of known transformations. The developer's problem is not inconsistency but rather gaps in domain knowledge about regulatory requirements. Examples cannot surface unknown compliance considerations.

WHERE THIS COMES FROM

Lesson 3.5: Iterative Refinement Techniques (Technique hierarchy)

Lesson 3.5: Iterative Refinement Techniques (When to use each technique)

Q28 · D3 3.4 PLAN-MODE-EXECUTION / HYBRID q-3-4-005 · Developer Productivity Tools CORRECT

A team is debugging a complex distributed system issue. The lead developer wants to use Claude Code to explore logs, trace request flows, and form hypotheses without making any changes to the codebase. Partway through the investigation, they identify a one-line fix in a configuration file and want to apply it immediately. What is the optimal workflow?

A Start in plan mode for investigation, then switch to direct execution for the one-line fix once identified

Plan mode is ideal for the investigation phase: exploring logs, tracing flows, and evaluating multiple hypotheses without committing to changes. Once the fix is identified and well-understood (a clear-scope, single-file change), switching to direct execution applies it efficiently without unnecessary planning overhead.

D Use allowedTools to restrict to read-only tools for investigation, then start a new session with write permissions for the fix

Starting a new session for the fix would lose all the investigation context (log analysis, request tracing, hypothesis formation) that led to identifying the fix. Switching modes within the same session preserves this valuable context.

B Use direct execution throughout, with detailed upfront instructions describing both the investigation process and potential fix patterns

Direct execution is inappropriate for the investigation phase because the problem and solution are not yet understood. Providing detailed upfront instructions assumes knowledge of the issue that the investigation is meant to discover.

C Use plan mode for the entire session, including the one-line fix, to maintain consistency

While plan mode is correct for investigation, continuing to use it for a well-understood one-line fix adds unnecessary overhead. Once the fix is identified and scoped, direct execution is the appropriate mode for a clear, limited change.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Hybrid plan then execute)

Q29 · D5 5.6 INFORMATION-PROVENANCE / CLAIM-SOURCE-MAPPING q-5-6-003 · Structured Data Extraction Pipeline

CORRECT

A data extraction pipeline extracts financial figures from annual reports. The pipeline cites sources using inline text references like 'According to page 12 of the 2024 Annual Report, revenue was \$4.2B.' When the extracted data is consumed by a downstream analytics system, the source attribution is consistently lost. What is the root cause and fix?

D Add a separate attribution database keyed by value hash that the downstream system queries to look up the source for any extracted figure.

While a database could store attributions, this adds unnecessary infrastructure when the extraction pipeline can simply output structured mappings directly. The attribution should travel with the data, not require a separate lookup.

A The downstream system strips text formatting. Fix by making the citations bold or using a special delimiter

The problem is not formatting but data structure. Regardless of text formatting, inline prose citations are lost when text is parsed, summarised, or restructured by downstream systems.

B Inline prose citations are fragile. Fix by outputting structured claim-source mappings pairing each value with source, page, and date.

Structured claim-source mappings survive downstream processing because they are data fields, not prose. An analytics system can consume {'value': '4.2B', 'source': '2024 Annual Report', 'page': 12} without losing attribution, whereas it will discard 'According to page 12...' during text processing.

C The downstream system needs to be modified to preserve all input text verbatim without any transformation

Requiring downstream systems to preserve verbatim text is impractical and fragile. Systems legitimately need to transform, aggregate, and restructure data. Attribution should be in a format that survives transformation.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Structured claim-source mappings)

Q30 · D4 4.6 MULTI-PASS-REVIEW / ATTENTION-DILUTION q-4-6-007 · Structured Data Extraction Pipeline

CORRECT

An extraction pipeline processes 50-page legal contracts in a single API call. It captures parties and dates from the first few pages but misses key financial terms in later pages, and occasionally contradicts itself across fields. A colleague suggests simply increasing the context window. What is the correct diagnosis and fix?

C Run the full extraction three times and take the most common value for each field

Multiple passes on the full document suffer from the same attention dilution pattern. The model will repeatedly extract early-page data well and miss later-page data.

D Add more detailed instructions about the importance of financial terms to increase the model's focus on those sections

Instructions cannot override the attention dilution effect of processing a 50-page document in one pass. The structural fix is breaking the extraction into focused per-section passes.

B This is attention dilution; split into per-section extraction passes plus a cross-section reconciliation pass.

Long documents cause attention dilution: the model attends thoroughly to early content but loses focus on later sections. Per-section passes ensure each part gets dedicated attention, and the integration pass catches contradictions and reconciles fields across sections.

A The context window is too small; upgrading to a larger context model will fix the inconsistency

If the document fits in the context window, a larger window does not improve attention quality. The problem is attention dilution over a long document, not context capacity.

WHERE THIS COMES FROM

Lesson 4.6: Multi-Instance Review and Output Validation (Multi-pass architecture)

Lesson 4.6: Multi-Instance Review and Output Validation (Why bigger context does not fix it)

Q31 · D3 3.1 CLAUDE-MD-HIERARCHY / MEMORY-COMMAND q-3-1-020

CORRECT

A developer's Claude Code applies the team's API conventions correctly in some sessions but not others, on the same project. They suspect the wrong memory files are loading in the failing sessions. What is the fastest way to confirm which CLAUDE.md and rules files a session has actually loaded?

D Ask Claude in the session to repeat the team's API conventions back

The model can paraphrase conventions from training or partial context, so a fluent answer does not prove the files loaded. `/memory` reports the loaded files deterministically instead of relying on the model's self-report.

C Delete `~/claude/CLAUDE.md` so only project-level configuration can load

Deleting user-level configuration is a destructive guess. It might mask the symptom, but it destroys the developer's personal setup without ever confirming what the failing sessions were loading.

B Run `/compact` to reload the configuration hierarchy from disk

`/compact` summarises the conversation to free context; it does not reload or report configuration files. `CLAUDE.md` content survives compaction, so this neither diagnoses nor fixes a loading difference.

A Run `/memory` in the session to list the loaded memory files

`/memory` is the diagnostic command for the configuration hierarchy: it shows which user-level, project-level, and directory-level memory files the current session has loaded, which directly confirms or rules out the suspected loading difference.

WHERE THIS COMES FROM

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (`/memory` and `CLAUDE.md` persistence)

Claude Code: Memory and CLAUDE.md ↗

Q32 · D3 3.2 SLASH-COMMANDS-SKILLS / SKILLS-FRONTMATTER q-3-2-002 · Developer Productivity Tools

CORRECT

A platform engineering team creates a `/security-audit` skill that scans the codebase for vulnerabilities. The skill should be available to every developer on the project, must not be able to modify any files, and produces extensive analysis output. Which configuration is correct?

A Place the skill in `~/claude/skills/` with `allowed-tools` restricting it to Read, Grep, and Glob, and `context: fork` in the frontmatter

`~/claude/skills/` is user-scoped and not shared via version control. The requirement states every developer on the project needs access, so it must be project-scoped.

B Place the skill in `.claude/commands/` with `allowed-tools` restricting it to `Read`, `Grep`, and `Glob`

While `.claude/commands/` is project-scoped (and equivalent to `.claude/skills/` — both paths create identical `/commands`), this configuration is missing context: `fork` for isolating the extensive analysis output. Frontmatter features like context: `fork` and `allowed-tools` require a `SKILL.md` file in `.claude/skills/`, making that the better location for this use case.

C Place the skill in `CLAUDE.md` as an always-loaded security scanning procedure

`CLAUDE.md` is for universal, always-loaded standards, not on-demand task-specific workflows. A security audit is invoked when needed, not applied to every session. Placing it in `CLAUDE.md` wastes tokens in sessions where no audit is needed.

D Place the skill in `.claude/skills/` with a `SKILL.md` containing `allowed-tools: ["Read", "Grep", "Glob"]` and context: `fork` in the frontmatter

`.claude/skills/` is project-scoped and shared via version control, so every developer gets it. `allowed-tools` restricts the skill to read-only tools, preventing file modifications. context: `fork` isolates the extensive analysis output from the main conversation. This satisfies all three requirements.

WHERE THIS COMES FROM

Lesson 3.2: Custom Slash Commands and Skills (Skills frontmatter)

Lesson 3.2: Custom Slash Commands and Skills (Project vs user scoping)

Claude Code: Skills ↗

Q33 · D1 1.2 ORCHESTRATION-PATTERNS / COORDINATOR q-1-2-002 · Multi-Agent Research System

CORRECT

A consulting firm's multi-agent research system has a coordinator that always invokes the full pipeline of five subagents (web search, document analysis, data extraction, synthesis, and formatting) for every query, including simple factual lookups that only need web search. This adds unnecessary latency and cost. What is the correct architectural fix?

A Have the coordinator select which subagents to invoke per query, not always the full pipeline.

The coordinator should analyse query requirements and decide which subagents are needed. A simple factual lookup only needs the web search agent, not the full five-agent pipeline. Dynamic selection reduces latency and cost for straightforward requests.

C Add a caching layer so subagents that have no work to do return immediately

Caching does not address the architectural issue. Invoking subagents that are not needed adds API call overhead and latency regardless of whether results are cached. The coordinator should not invoke unnecessary subagents in the first place.

B Allow subagents to communicate directly with each other so they can skip unnecessary steps in the pipeline

Direct subagent communication violates the hub-and-spoke architecture. All communication must flow through the coordinator. The fix is smarter coordinator routing, not bypassing the coordinator.

D Split into two separate systems: one for simple queries and one for complex queries

Maintaining two separate systems creates unnecessary complexity. The coordinator's role is precisely to analyse query requirements and make routing decisions. A single coordinator with dynamic selection handles both cases.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q34 · D5 5.3 ERROR-PROPAGATION / SILENT-SUPPRESSION q-5-3-006 · Enterprise Data Platform with Federated Queries

CORRECT

The data platform agent queries three sources in sequence: Snowflake (succeeds), PostgreSQL (returns a connection timeout), and a pricing API (succeeds). The agent silently skips the PostgreSQL failure and produces a report using only Snowflake and API data, without informing the user that customer data is missing. What is the critical design failure?

B The agent silently suppresses the error instead of propagating structured error context that annotates which data sources failed and what coverage the report actually has.

Silent error suppression is one of the most dangerous anti-patterns in multi-source systems. The agent must propagate structured error context — identifying which sources failed, what data is missing, and what coverage the final report

represents — so the user can make informed decisions about the report's completeness.

C The agent should abort the entire report if any single data source fails, to avoid producing incomplete results.

Aborting on any failure is overly conservative. Partial results with clear coverage annotations are often more useful than no results. The fix is transparency about what is missing, not refusing to produce anything.

D The orchestrator should pre-check all data source connections before allowing the agent to begin querying.

Pre-checking connections does not prevent runtime failures (a source can become unavailable between the check and the query). The agent must handle failures gracefully at query time with structured error propagation.

A The agent should retry the PostgreSQL query automatically with exponential backoff before producing the report.

Automatic retries are appropriate for transient errors, but the critical failure is that the agent silently suppresses the error. Even after retries exhaust, the agent must surface the failure to the user rather than producing an incomplete report without disclosure.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Silent suppression anti-pattern)

Lesson 5.3: Error Propagation in Multi-Agent Systems (Structured error context)

Q35 · D4 4.3 STRUCTURED-OUTPUT / TOOL-CHOICE-FORCING q-4-3-006 · Structured Data Extraction Pipeline **CORRECT**

Your data extraction pipeline uses `tool_use` with `tool_choice` set to 'auto'. You need guaranteed structured output for every request, but some documents occasionally produce conversational text responses instead. You have only one extraction tool defined. What is the most reliable fix?

B Set `tool_choice` to force the specific extraction tool by name

Setting `tool_choice` to `{type: 'tool', name: 'extract_data'}` forces the model to call that specific tool on every request, guaranteeing schema-compliant structured output with no possibility of text-only responses.

C Add a post-processing step that retries any request returning text instead of a tool call

Retrying with the same 'auto' `tool_choice` may produce the same text response. Fixing `tool_choice` eliminates the problem at the API level rather than patching it after the fact.

D Switch to prompt-based JSON extraction with strict formatting instructions for more predictable output

Prompt-based JSON is less reliable than `tool_use`. It risks syntax errors and missing fields. Forcing the tool via `tool_choice` is the correct approach.

A Add a system prompt instruction: 'You must always use the extraction tool and never produce text responses'

System prompt instructions cannot guarantee tool use when `tool_choice` is 'auto'. The API-level `tool_choice` parameter is the authoritative mechanism.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (`tool_choice` modes)

Anthropic: Tool Use Documentation ↗

Q36 · D1 1.2 ORCHESTRATION-PATTERNS / COORDINATOR q-1-2-013 · Multi-Agent Research System **CORRECT**

In a hub-and-spoke research system, a coordinator manages several search and analysis subagents. Which responsibilities sit with the coordinator rather than with individual subagents? (Select 3)

D Running every search itself first so subagents only verify its findings.

The coordinator delegates work rather than duplicating it; doing every search itself defeats the point of decomposition.

B Deciding which subagents to invoke based on the complexity of each incoming query.

The guide places dynamic subagent selection with the coordinator, which analyses query requirements instead of always running the full pipeline.

E Aggregating subagent results and re-delegating targeted follow-up queries when synthesis shows gaps.

Result aggregation and gap-driven re-delegation are coordinator responsibilities in the iterative refinement loop.

C Routing all inter-agent communication so subagents never exchange messages directly.

Hub-and-spoke routes every message through the coordinator for observability and consistent error handling.

A Sharing its full conversation history with each subagent automatically at spawn time.

Subagents run with isolated context; nothing is inherited automatically, and the coordinator passes only the context each task needs.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Lesson 1.2: Multi-Agent Orchestration (Hub-and-spoke architecture)

Q37 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ACCESS-VS-EMPTY q-2-2-008 ·

Enterprise Data Platform with Federated Queries

CORRECT

The data platform's `fetch_api` tool calls a third-party pricing API. When the API key has expired, the tool returns `{ "data": [], "status": "ok" }` instead of signalling an authentication failure. The agent tells the user 'No pricing data is available for that product' and moves on. What is the root cause?

C The third-party API is poorly designed. The platform team should switch to a different pricing provider that returns proper HTTP error codes.

Whilst the upstream API behaviour is unhelpful, the platform team controls the MCP tool layer. The tool should detect the authentication failure (even from the upstream response) and translate it into a structured MCP error rather than passing through the misleading response.

B The tool cannot distinguish an access failure (expired key) from a valid empty result, so the agent treats a permission error as no data.

The tool returns the same structure for two fundamentally different outcomes: 'I could not access the data' versus 'I accessed the data and found nothing.' The agent cannot distinguish these and accepts the empty result at face value. The tool must return a structured error with the MCP `isError` flag for authentication failures.

A The agent should be instructed via the system prompt to treat empty arrays as errors and retry with exponential backoff.

Not all empty arrays are errors — a product with no pricing data is a valid empty result. The problem is that the tool masks an authentication failure as a successful empty response. System prompt instructions cannot reliably distinguish the two.

D The agent's context window is too small to hold the full API response, so it receives a truncated version that appears empty.

Context window truncation would not produce a well-formed `{ "data": [], "status": "ok" }` response. The problem is semantic — the tool is returning a misleading success status for an authentication failure.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

Q38 · D5 5.6 INFORMATION-PROVENANCE / CONFLICT-HANDLING q-5-6-001 · Multi-Agent Research System

CORRECT

A multi-agent research system produces a synthesis report on market trends. Two credible sources report different growth rates for the same sector: Source A reports 12% growth (2023 data) and Source B reports 8% growth (2024 data). The synthesis agent currently selects the more recent value. What is the correct approach?

A Always use the most recent source as it reflects the latest data

This silently discards valid historical context. The difference may reflect an actual trend, not a contradiction.

B Average the two values and report 10% growth with a note about source variance

Averaging conflicting statistics is mathematically misleading and destroys the temporal context that explains the difference.

D Flag the conflict and escalate to a human researcher for resolution before including in the report

This is unnecessary — the values are not contradictory, they reflect different time periods. Proper annotation resolves the ambiguity.

C Annotate both values with source and publication dates, letting the consumer interpret them.

This preserves provenance, temporal context, and both data points. The consumer can see that different dates explain different numbers.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflict handling)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Temporal awareness)

Q39 · D2 2.4 MCP-SERVER-INTEGRATION / CONFIG-SCOPE q-2-4-008 · Enterprise Data Platform with Federated Queries **CORRECT**

The data platform team needs to share their Snowflake and PostgreSQL MCP server configurations with all team members whilst allowing individual developers to experiment with a personal API integration server. Where should each configuration be placed?

B Place the Snowflake and PostgreSQL servers in the project-level `.mcp.json`, and personal API integration servers in user-level `~/.claude.json`.

Project-level `.mcp.json` is version-controlled and shared — correct for team-wide database integrations. User-level `~/.claude.json` is machine-specific — correct for personal experimental servers that should not affect other team members.

C Place all three servers in `~/.claude.json` and have each developer copy the configuration manually.

User-level configuration is not version-controlled and requires manual synchronisation. Team-wide servers belong in project-level `.mcp.json` for consistency and change tracking.

A Place all three servers in the project-level `.mcp.json` so they are version-controlled and consistent across the team.

Personal experimental servers should not be in project-level configuration. Changes to personal servers would create unnecessary merge conflicts and force experimental integrations on the entire team.

D Place the database servers in environment variables and reference them in both `.mcp.json` and `~/.claude.json`.

Environment variables are used for credentials (connection strings, API keys), not for MCP server configuration. The server definitions themselves belong in `.mcp.json` or `~/.claude.json` depending on scope.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Scoping hierarchy)

Claude Code: MCP Server Configuration ↗

Q40 · D1 1.5 AGENT-SDK-HOOKS / PRETOOLUSE-HOOKS q-1-5-013 · Developer Productivity Tools **CORRECT**

A developer-tool agent writes code and runs shell commands. A system prompt instruction ('Do not run destructive commands or access files outside the project') is violated 3% of the time in testing. The team requires this rule to hold absolutely. What should the architect implement?

A Strengthen the system prompt with more specific examples of forbidden commands and add few-shot demonstrations

The prompt already fails 3% of the time. Strengthening it may reduce the failure rate but cannot eliminate it. Destructive operations require 100% prevention, which only deterministic enforcement can guarantee.

B PreToolUse hooks that scan shell commands for destructive patterns and validate file paths before execution.

PreToolUse hooks intercept tool calls before execution. A hook can pattern-match against destructive commands like 'rm -rf' and validate that file paths fall within the project directory, blocking any violation with 100% reliability. This is deterministic enforcement for a safety-critical requirement.

C Implement PostToolUse hooks that detect and roll back destructive commands after they execute

PostToolUse hooks run after execution. By the time a PostToolUse hook detects 'rm -rf', the files are already deleted. Destructive commands cannot be reliably rolled back. Prevention via PreToolUse is required, not post-execution detection.

D Remove the shell execution tool entirely to prevent any possibility of destructive commands

Removing shell execution eliminates the agent's ability to run any commands, including legitimate ones needed for its development workflow. The correct approach is surgical — block specific dangerous patterns while allowing legitimate usage.

WHERE THIS COMES FROM

Lesson 1.5: Agent SDK Hooks (PreToolUse hooks)

Anthropic: Agent SDK Hooks ↗

Q41 · D1 1.2 ORCHESTRATION-PATTERNS / DECOMPOSITION q-1-2-001 · Multi-Agent Research System

CORRECT

A multi-agent research system produces a report on 'renewable energy technologies' that only covers solar and wind, missing geothermal, tidal, biomass, and nuclear fusion. Each subagent produced thorough coverage of its assigned topic. Where is the root cause?

B The synthesis subagent failed to identify gaps in the research

The synthesis agent can only work with the research it receives. It cannot synthesise topics that were never researched.

A The web search subagent's search queries were too narrow

The web search subagent researched what it was assigned. The issue is what it was assigned, not how it searched.

C The coordinator's task decomposition assigned only solar and wind, omitting the other renewable categories.

The coordinator is responsible for decomposing the broad topic into subtopics. If it only assigns solar and wind, no downstream agent can cover the missing categories. Trace failures to their origin.

D The document analysis subagent lacked access to sources on geothermal, tidal, biomass, and fusion, so those categories never entered the report.

Source availability is not the root cause. The coordinator never asked any agent to research geothermal, tidal, biomass, or fusion, so no fix to source access would help.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Narrow decomposition failure)

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q42 · D4 4.3 STRUCTURED-OUTPUT / NULLABLE-FIELDS q-4-3-001 · Structured Data Extraction Pipeline

CORRECT

Your extraction system uses tool_use with a strict JSON schema. All fields are marked as required. Testers report that the model invents plausible-looking dates and amounts when processing documents that lack this information. What is the best fix?

B Switch from tool_use to prompt-based JSON extraction

This moves backwards in reliability. Prompt-based JSON introduces syntax errors without solving the fabrication problem.

D Add a validation step that checks all values against the source document

Post-hoc validation is useful but does not address the root cause. Making fields optional prevents fabrication at the schema level.

A Add an instruction telling the model not to hallucinate

Vague instructions do not override the schema constraint. Required fields pressure the model to produce values.

C Make fields optional/nullable when source documents may not contain the information

Optional/nullable fields allow the model to return null instead of fabricating values. This directly prevents fabrication for missing information.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (Nullable / optional fields)

Q43 · D4 · 4.4 VALIDATION-RETRY / INTER-STEP-VALIDATION · q-4-4-006 · Structured Data Extraction Pipeline CORRECT

Your document processing pipeline chains three steps: (1) extract raw text, (2) classify document type, (3) extract structured fields based on type. Step 2 occasionally misclassifies invoices as purchase orders, causing step 3 to extract the wrong fields. The team proposes combining steps 2 and 3 into a single prompt to reduce errors. What is the better approach?

A Combine steps 2 and 3 as proposed, since fewer steps means fewer failure points

Combining steps creates a less focused prompt that must handle both classification and extraction simultaneously. This increases attention dilution and makes it harder to diagnose which part fails.

B Keep steps separate and validate the classification between steps 2 and 3 before extraction

Keeping steps separate maintains focused prompts. Adding validation between steps catches misclassifications before they cascade into wrong-field extraction. This is the core principle of prompt chaining: focused steps with inter-step validation.

D Run step 2 three times and use majority voting to determine the document type

Majority voting adds cost and latency without addressing the root cause. Validation between steps is more efficient and catches the specific failure mode.

C Add more few-shot classification examples to step 2's prompt so misclassifications stop occurring

Better examples may help, but without validation between steps, misclassifications still cascade silently into step 3. Inter-step validation is the structural fix.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Self-correction flow)

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry boundary)

Q44 · D4 · 4.3 STRUCTURED-OUTPUT / NULLABLE-FIELDS · q-4-3-007 · Structured Data Extraction Pipeline CORRECT

An extraction pipeline uses 'tool_use' with a 15-required-field schema. For some document types, only 8 of those fields ever appear in the source material. The model consistently fabricates plausible values for the missing 7, and the instruction 'do not fabricate data' has not helped. Should the team (A) keep all fields required and add post-extraction validation, or (B) redesign the schema? Which is correct and why?

C Create separate schemas for each document type, each containing only the fields present in that document type

Multiple schemas add maintenance complexity and require document type classification before extraction. Making fields nullable in a single schema is simpler and equally effective.

B Redesign the schema to make the 7 sometimes-absent fields optional/nullable, allowing the model to return null instead of fabricating

Making fields optional/nullable when source data may be absent removes the schema-level pressure to fabricate. The model can legitimately return null for missing information, which is a correct representation of reality. This addresses the root cause at the schema level.

D Switch to prompt-based JSON extraction where you can instruct the model to omit absent fields entirely

Prompt-based JSON sacrifices schema compliance guarantees. tool_use with optional fields gives both flexibility for absent data and schema validation for present data.

A Keep all fields required and add post-extraction validation to detect and remove fabricated values

Post-extraction validation cannot reliably distinguish fabricated values from correct ones — fabricated values are designed to look plausible. The schema itself forces fabrication by making all fields required.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (Nullable fields)
Lesson 4.2: Structured Output with Tool Use (Schema does not prevent fabrication)

Q45 · D5 5.5 HUMAN-REVIEW-CALIBRATION / STRATIFIED-SAMPLING q-5-5-009 · Structured Data Extraction Pipeline

CORRECT

Your extraction pipeline reports 97% aggregate accuracy, and you want to automate high-confidence extractions. Which safeguards should be in place before you reduce human review? (Select 3)

C

Verify accuracy separately by document type and field segment, not just in aggregate.

A strong aggregate figure can mask poor performance on specific document types or fields.

D

Keep stratified random samples of high-confidence extractions under review to measure error rates and catch novel patterns.

Stratified sampling is the guide's mechanism for ongoing measurement after automation.

E

Expand the review team until every extraction can be checked by hand.

Reviewing everything ignores the point of calibration: routing limited reviewer capacity to low-confidence and ambiguous cases.

B

Rely on the aggregate accuracy figure once it has held steady for a full quarter.

Stability of the aggregate number does not reveal per-segment weaknesses, which is the risk the guide highlights.

A

Calibrate field-level confidence thresholds against a labelled validation set.

Confidence scores only route review attention correctly once they are calibrated on labelled data.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Aggregate metrics trap)

Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

Lesson 5.5: Human Review and Confidence Calibration (Field-level calibration)

Q46 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / CONTEXT-PASSING q-1-3-015 · Multi-Agent Research System

CORRECT

A coordinator spawns a research subagent to investigate market trends. The subagent returns detailed findings. The coordinator then needs to pass these findings to a synthesis subagent for report writing. However, the synthesis subagent produces a report that contradicts the research findings. What is the most likely cause?

C

The synthesis subagent has its own web search tool and keeps pulling contradictory information from the open web instead of the research it was handed.

A synthesis subagent should not have web search tools; that violates scoped tool access. The real issue is that the full research output is not being passed to it, not that it is searching the web.

D

The two subagents are using different model versions with different training data

Model version differences would not cause contradictions with explicitly provided research data. The issue is that the research data is not being fully passed to the synthesis agent, not that the agents have different base knowledge.

B

The coordinator passes a summary instead of the full structured research output, so the synthesis agent fills gaps from its training data.

Subagents do not share memory. The coordinator must explicitly pass the complete research output to the synthesis agent. If the coordinator only passes a summary, the synthesis agent has no access to the detailed findings and will rely on its training data to fill gaps, potentially contradicting the actual research.

A

The synthesis subagent has a conflicting system prompt that overrides the research data

While possible, conflicting system prompts would produce consistently wrong outputs across all requests. The more common cause is that the coordinator is not passing the full research output, so the synthesis agent fills gaps with its own training data.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Context passing)

Lesson 1.3: Subagent Invocation and Context Passing (Structured metadata format)

Q47 · D2 2.3 TOOL-DISTRIBUTION-CHOICE / CONSOLIDATION q-2-3-008 · Enterprise Data Platform with Federated Queries CORRECT

The data platform team has built an MCP server with 22 tools: `query_snowflake`, `query_postgres`, `query_api`, plus 19 specialised tools for individual data transformations (`pivot_table`, `calculate_percentile`, `normalise_currency`, etc.). Agents take 3-4 turns to select the correct tool and frequently choose the wrong transformation. What is the most effective redesign?

C Use `tool_choice: 'any'` to force the agent to always call a tool, eliminating turns where the agent reasons without acting.

Forcing tool calls does not improve selection accuracy — it just ensures the agent picks something, potentially the wrong tool. The root cause is too many similar tools, not too few tool calls.

B Consolidate the 19 transformation tools into a single `transform_data` tool with a `transform_type` parameter, reducing the total to 4 tools.

Reducing from 22 to 4 tools brings the count within the optimal 4-5 range for reliable tool selection. The 19 transformation tools share a common pattern (input data, transformation type, output format) and are natural candidates for consolidation into a single parameterised tool.

A Improve all 22 tool descriptions with detailed examples and boundary conditions to help the agent distinguish between them.

Better descriptions help, but 22 tools still exceeds the practical limit for reliable selection. Research shows tool selection degrades significantly beyond 4-5 tools per agent. The tool count itself is the problem.

D Split the tools across two separate MCP servers — one for queries and one for transformations — to reduce cognitive load.

MCP server boundaries are invisible to the agent. All tools from all connected servers appear in a single list. Splitting across servers does not reduce the number of tools the agent must choose from.

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (Consolidating near-duplicate tools)

Lesson 2.3: Tool Distribution and Tool Choice (Tool overload)

Q48 · D3 3.5 ITERATIVE-REFINEMENT / BATCH-VS-SEQUENTIAL q-3-5-002 · Developer Productivity Tools INCORRECT

A developer is iterating on a data transformation function with Claude Code and has identified three issues: (1) date parsing mishandles timezone offsets, (2) currency formatting uses the wrong locale, and (3) the output JSON schema has an incorrect field name. Issues 1 and 2 are independent; issue 3 changes the output shape both must conform to. How should the developer provide feedback?

B Send each issue in a separate sequential message, waiting for confirmation after each fix

Pure sequential iteration is appropriate when all issues are independent. Here, issue 3 (schema field name) affects the expected output that issues 1 and 2 must conform to. Fixing issues 1 and 2 first without correcting the schema would produce fixes targeting the wrong field name.

C Fix issue 3 (schema field name) first, then address issues 1 and 2 one at a time since they are independent

Issue 3 changes the output schema that issues 1 and 2 must conform to, so it must be resolved first. Once the schema is correct, issues 1 and 2 are independent, so they are fixed one at a time rather than batched, since batching independent issues can confuse which feedback applies to which fix. This follows the principle: resolve dependencies first, batch when fixes interact, and sequence when independent. (correct answer)

D Use the interview pattern to have Claude ask clarifying questions about all three issues before making any changes

The interview pattern is for unfamiliar domains where the developer might miss considerations. Here, the developer has already identified the three specific issues and understands their interdependencies. The question is about feedback sequencing, not domain exploration. (your answer)

A Send all three issues in a single message to let Claude address them holistically

Batching everything together is appropriate when all issues are interdependent. Here, issues 1 and 2 are independent of each other, and sending them together risks conflating the feedback for those fixes. The interdependency is only between issue 3 and the other two.

WHERE THIS COMES FROM

Lesson 3.5: Iterative Refinement Techniques (Batch vs sequential)

Q49 · D2 2.5 BUILT-IN-TOOLS / GREP-VS-GLOB q-2-5-008 · Enterprise Data Platform with Federated Queries CORRECT

A developer is investigating a data inconsistency in the platform codebase. They need to find all files that reference a specific Snowflake table name 'fact_revenue_daily' and then check the directory structure for related migration files. Which tool sequence is correct?

C Grep for 'fact_revenue_daily' to find all content references, then Glob for '**/migrations/*.sql' to find migration files by path pattern.

Grep searches file contents — correct for finding code that references the table name. Glob matches file paths — correct for finding migration files by directory pattern. This is the optimal two-step sequence using each tool for its strength.

D Grep for 'fact_revenue_daily' for both steps — first to find references, then to find migration files containing the table.

Using Grep for both steps would find migration files that mention the table, but would miss migration files related to the table that do not contain its name (e.g. schema migrations, index changes). Glob is the correct tool for path-based file discovery.

A Glob for '**/*fact_revenue_daily*' to find references, then Glob for '**/migrations/**' to find migration files.

Glob matches file paths by name, not file contents. It would find files named after the table (unlikely) but miss all files that reference the table name in their code.

B Read all SQL files in the project to search for the table name, then Read all migration files.

Reading all files upfront is a context-budget anti-pattern. It wastes tokens on irrelevant files and is exactly the approach the exam penalises.

WHERE THIS COMES FROM

Lesson 2.5: Built-in Tools (Grep vs Glob)

Q50 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / PARALLEL-SPAWNING q-1-3-002 · Multi-Agent Research System CORRECT

A multi-agent research system has a coordinator that spawns a web search subagent and a document analysis subagent sequentially across separate API turns. The web search completes in 8 seconds and the document analysis completes in 12 seconds, giving a total latency of 20 seconds. The two subagents are investigating independent topics and do not depend on each other's results. How should the architect reduce this latency?

B Have the coordinator emit both Task tool calls in a single response to spawn both subagents in parallel, reducing total latency to roughly 12 seconds

When subagents investigate independent topics, the coordinator can emit multiple Task tool calls in a single response. Both subagents run concurrently, so total latency is determined by the slower one (12 seconds) rather than the sum (20 seconds).

C Allow the web search subagent to directly invoke the document analysis subagent after it finishes, removing the coordinator round-trip

Direct subagent communication violates the hub-and-spoke architecture where all communication flows through the coordinator. This also would not achieve parallel execution — it would still be sequential.

A Merge the web search and document analysis into a single subagent to reduce coordination overhead

Merging agents violates the principle of specialised subagents with scoped tool access. It also does not reduce total processing time — the merged agent still needs to do both tasks, and loses the benefit of parallel execution.

D Use fork_session to split the coordinator into two parallel branches, one for each subagent

fork_session is for divergent exploration of the same problem from a shared baseline, not for parallel subagent invocation. Emitting multiple Task tool calls in a single coordinator response is the correct mechanism for spawning concurrent subagents.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning)

Lesson 1.3: Subagent Invocation and Context Passing (Task tool)

Q51 · D4 · 4.4 VALIDATION-RETRY / PYDANTIC-VALIDATION · q-4-4-010 · Structured Data Extraction Pipeline CORRECT

The pipeline validates each tool_use extraction against a Pydantic model with model_validate(). Validation succeeds and every field has the correct type, yet extracted totals still disagree with the documents' stated totals. What explains this, and what is the fix?

D Semantic disagreement indicates hallucination; lower the temperature so extraction becomes deterministic

Sampling settings do not create correctness, and determinism would only repeat the same wrong extraction; this failure class belongs to semantic validation.

A model_validate() is failing silently; pin an older Pydantic version until the regression is fixed

model_validate() did its job: the extraction conforms to the model's types. Nothing failed silently; the missing check was never defined.

B Type checks passed but no rule relates the totals; add a model validator for the sum rule and route its errors into retries

Schema conformance and semantic correctness are different layers; the sum relationship must be written as a validator, whose errors drive the retry loop.

C The schema needs numeric minimum and maximum constraints on the stated_total field so that mismatched totals are rejected at parse time

A per-field range cannot express a relationship between two fields; the mismatch is only detectable by comparing them in validation code.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Pydantic as the validation layer)

Q52 · D4 · 4.4 VALIDATION-RETRY / RETRY-WITH-ERROR · q-4-4-002 · Structured Data Extraction Pipeline CORRECT

Your extraction pipeline retries failed documents, but the retry simply resends the original document with the same prompt. Success rates on retries are only marginally better than the first attempt. Developers have identified that most failures are format mismatches where the model places values in the wrong fields. How should you improve the retry mechanism?

A Increase the number of retries from 1 to 3, since extraction is non-deterministic and more attempts improve odds

Naive retries without error feedback produce the same mistakes. More attempts at the same flawed approach yield diminishing returns. The model needs to see what went wrong.

D Send the original document, the failed extraction, and the validation error naming the misplaced fields.

Retry-with-error-feedback is dramatically more effective than naive retries. Sending the original document, the failed extraction, and the specific validation error allows the model to self-correct. Format mismatches are fixable errors that respond well to this approach.

B Switch to a different model for retries so a fresh perspective catches the errors

The issue is not the model but the absence of error context. Any model retrying without seeing the specific validation error will likely repeat the same mistakes.

C Skip retries and route all failures directly to human review to avoid wasting API costs

Format mismatches are fixable errors. The model can self-correct when given proper error feedback. Skipping retries wastes the model's self-correction capability for errors that are resolvable.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry with error feedback)

Lesson 4.4: Validation, Retry, and Feedback Loops (Self-correction flow)

Q53 · D4 · 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-coverage · q-4-2-001 · Structured Data Extraction Pipeline CORRECT

Your extraction pipeline correctly identifies data in structured tables but returns empty fields when the same information appears in narrative paragraphs. Detailed instructions specify all required fields. What should you try first?

D

Add a post-processing retry that re-extracts any empty fields

Retrying without better guidance will produce the same empty results. The model needs examples of correct extraction from narrative text.

B

Add few-shot examples showing correct extraction from both structured tables and narrative paragraphs

Few-shot examples demonstrating correct handling of varied document structures directly address inconsistent extraction quality across formats.

A

Increase the model's context window to process more of the document

The model already finds data in tables. The issue is not context size but inconsistent handling of different document structures.

C

Add a pre-processing step to convert all narrative text into tables

This adds unnecessary complexity. The model should handle varied formats — few-shot examples teach it how.

WHERE THIS COMES FROM

Lesson 4.2: Few-Shot Prompting (Few-shot covering varied input)

Lesson 4.2: Structured Output with Tool Use (What tool_use does not enforce)

Q54 · D1

1.2 ORCHESTRATION-PATTERNS / COORDINATOR

q-1-2-005 · Multi-Agent Research System

CORRECT

A research coordinator spawns 4 subagents to investigate different aspects of a market analysis. Agent A fails with an API error, Agent B returns partial results (3 of 5 requested data points), and Agents C and D return complete results. The final report is due in 2 hours. What should the coordinator do?

A

Retry Agent A, discard Agent B's partial results as unreliable, and wait for both agents to return fully complete data before writing the report.

Discarding Agent B's partial results wastes valid data, and waiting for full completion ignores the two-hour deadline. The coordinator should use what is available while attempting to fill gaps.

B

Proceed with only Agents C and D's results and produce the report without the data from Agents A and B

This unnecessarily discards Agent B's partial results and abandons Agent A's scope entirely. A thorough coordinator should attempt recovery while incorporating all available data.

D

Terminate all agents and restart the entire investigation from scratch with a simplified scope

Restarting discards the complete results from Agents C and D and the partial results from Agent B. This wastes time and compute when most of the investigation has already succeeded.

C

Retry Agent A, incorporate Agent B's partial results, flag B's missing data points as gaps, and synthesise from all available results.

The coordinator should maximise data utilisation: retry the failed agent (A), incorporate partial results from B while flagging the 2 missing data points as known gaps, and synthesise all available results from A (after retry), B (partial), C, and D. Explicit gap annotations ensure the report is transparent about its completeness.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Lesson 1.7: Session State and Resumption (Recovering from partial results)

Q55 · D3

3.1 CLAUDE-MD-HIERARCHY / PRECOMPACT-HOOK

q-3-1-018

CORRECT

A documentation team wants to archive the full conversation transcript to a log file every time Claude Code runs `/compact`, so that context lost during compaction can be reviewed later. Which hook configuration achieves this?

A A PostToolUse hook on Write that snapshots the transcript whenever a file is written, on the assumption /compact will eventually run

Write and compaction are unrelated. PostToolUse on Write would fire on every file write whether or not /compact is about to run, producing noise rather than a transcript at the right moment. The dedicated PreCompact event is the correct hook.

D A PostToolUse hook on all tools that appends each tool result to a running log file, creating a continuous archive

Logging every tool result creates enormous log files and does not capture conversation context (reasoning, planning, user messages). The targeted PreCompact event captures the full transcript at the critical moment without per-tool noise.

C A PreToolUse hook configured on a built-in 'Compact' tool, matching tool name 'Compact'

There is no 'Compact' tool exposed to PreToolUse. Compaction is a Claude Code lifecycle operation, not a model-invoked tool, so it has its own dedicated PreCompact event rather than being hooked via PreToolUse.

B A PreCompact hook that writes the current transcript to a timestamped log file before /compact summarises the conversation

PreCompact is a first-class hook event in Claude Code that fires immediately before /compact (or auto-compaction) runs. The hook receives the full pre-compaction transcript path and can copy or process it before summarisation discards detail. This is the documented mechanism for exactly this use case.

WHERE THIS COMES FROM

Claude Code: Hooks ↗

Q56 · D4 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-AMBIGUITY q-4-2-007 · Structured Data Extraction Pipeline CORRECT

An invoice-extraction pipeline mislabels fields on documents where the correct interpretation is ambiguous. Which few-shot practices should you apply? (Select 2)

D Create two to four targeted examples for the ambiguous cases, showing why one interpretation is chosen.

Targeted examples that demonstrate the reasoning are the guide's specific remedy for ambiguous cases.

C Replace the examples with a longer natural-language description of each field.

Descriptions alone are what produced the inconsistency; examples communicate the expected mapping more reliably than more prose.

A Add an example for every document variant seen in production so far.

Volume is not the fix; a handful of targeted examples for the ambiguity beats exhaustive coverage and its context cost.

B Use examples showing correct extraction from varied document structures to fix empty or null extraction of required fields.

Varied-structure examples fix empty or null extraction of fields that are present, which is the few-shot remedy for unreliable extraction; fabrication of absent data is handled by nullable schema fields instead.

WHERE THIS COMES FROM

Lesson 4.2: Few-Shot Prompting (Few-shot construction)

Anthropic: Multishot (Few-Shot) Prompting ↗

Q57 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / SCOPED-TOOLS q-1-3-016 · Multi-Agent Research System CORRECT

A coordinator delegates to a data collection agent ('web_search', 'database_query') and an analysis agent ('calculate', 'chart_generate'). In testing, the analysis agent calls 'web_search' directly to fetch extra context, bypassing the data collection agent. What is the architectural violation and how should it be fixed?

C Allow the analysis agent to keep web_search access since it sometimes needs additional context for better analysis

This violates scoped tool access and bypasses the data collection agent's role. If the analysis agent needs more data, the correct flow is: analysis agent signals the coordinator, coordinator delegates to the data collection agent, results flow back through the coordinator.

B Add a system prompt instruction telling the analysis agent not to use `web_search` even though it has access

Prompt instructions are probabilistic. The agent is already occasionally using `web_search` despite the architectural intent. The fix is to remove the tool from `allowedTools`, providing deterministic enforcement rather than hoping the prompt works.

A The analysis agent's `allowedTools` includes `web_search`; remove it so only the data collection agent has it.

Each agent should only have tools relevant to its specific role. The analysis agent should have `calculate` and `chart_generate`, not `web_search`. If the analysis agent needs additional data, it should request it through the coordinator, which then delegates to the data collection agent. This enforces the scoped tool access principle.

D Implement a `PostToolUse` hook that logs when the analysis agent uses `web_search` for monitoring purposes

Logging the violation does not prevent it. The architectural fix is to remove `web_search` from the analysis agent's `allowedTools`, not to monitor its misuse.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Scoped tool access)

Anthropic: Claude Agent SDK Overview ↗

Q58 · D5 5.6 INFORMATION-PROVENANCE / CLAIM-SOURCE-MAPPING q-5-6-002 · Multi-Agent Research System **CORRECT**

A multi-agent research system has three subagents (financial filings, news, technical white papers) and a synthesis agent. Each subagent returns properly attributed findings, but the final synthesised report has no source attribution: stakeholders cannot trace which claim came from which source. Which fix addresses the root cause?

B Have each subagent include source URLs as inline hyperlinks in their prose output so the synthesis agent can preserve them

Inline hyperlinks in prose are fragile — the synthesis agent will rewrite, merge, and compress prose during summarisation, stripping or disconnecting links from their associated claims. Attribution must be in structured data, not embedded in prose that gets rewritten.

A Append a bibliography section at the end of the report listing all sources each subagent consulted

A bibliography lists sources but does not map specific claims to specific sources. Stakeholders need to know which claim came from which document, not just which documents were consulted overall. This is document-level attribution, not claim-level provenance.

C Require subagents to output structured claim-source mappings and instruct the synthesis agent to preserve and merge them.

Structured claim-source mappings survive synthesis because they are data structures, not prose that gets rewritten. The synthesis agent can merge mappings from multiple subagents while preserving the link between each claim and its source. This also enables content-appropriate rendering: financial data as tables, news as prose, technical findings as lists — each with attribution intact.

D Store all subagent outputs in a database and have the synthesis agent reference database entries by ID instead of incorporating content directly

This adds infrastructure complexity without solving the synthesis problem. The synthesis agent still needs to merge and present findings coherently. Database references do not prevent attribution loss during the summarisation and rewriting that synthesis inherently requires.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Structured claim-source mappings)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Attribution preservation)

Q59 · D2 2.1 TOOL-SCHEMA-DESIGN / SPLITTING q-2-1-008 · Developer Productivity Tools **INCORRECT**

A developer productivity platform has a tool called `analyze_content` with the description 'Analyses content from various sources.' Logs show the agent uses it for web scraping, document parsing, and code analysis indiscriminately, leading to poor results for each use case. What is the most effective fix?

B Improve the tool description to list all supported content types and their expected formats.

A better description on a generic tool does not change the fact that the tool's processing logic is too broad. When a tool serves multiple distinct purposes, the fix is to split it into purpose-specific tools. (your answer)

A Add detailed few-shot examples to the system prompt showing which types of content the tool handles well and which it does not.

Few-shot examples add token overhead and treat the symptom. The root cause is that a single generic tool is trying to serve three distinct use cases, each requiring different processing logic.

C Split `analyze_content` into purpose-specific tools: `extract_web_results`, `parse_document`, and `analyze_code`, each with a narrow remit.

Tool splitting replaces a generic tool with purpose-specific alternatives that each have a clear, narrow description and defined input/output contracts. Each resulting tool handles one use case well, which eliminates ambiguity about when to use it. (correct answer)

D Implement a routing layer that inspects the input content type and dispatches to the appropriate internal processing function within `analyze_content`.

An internal routing layer hides the content type distinction from the agent. Splitting into separate tools exposes the distinction through tool descriptions, enabling the LLM's natural language understanding to select the right tool.

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Tool splitting)

Lesson 2.1: Tool Interface Design (Tool descriptions)

Q60 · D4 4.4 VALIDATION-RETRY / RETRY-BOUNDARY q-4-4-008 · Structured Data Extraction Pipeline

CORRECT

A validation-retry loop still fails on the same documents after three attempts each. In which situations will further retries not help? (Select 2)

B The required information is simply absent from the source document.

No amount of retrying makes missing information appear; the guide marks absent data as the boundary of retry effectiveness.

D The information exists only in an external document that is not provided in context.

Retries cannot reach data outside the model's context; the fix is supplying the external document, not another attempt.

C The validation service timed out on the last attempt.

A timeout is a transient failure, which is precisely the class of error a retry is likely to resolve.

A The model placed a correct value in the wrong field on the last attempt.

A value in the wrong field is a semantic validation error; retry-with-error-feedback naming the misplaced field guides the next attempt to correct it.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry boundary)

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry with error feedback)

SUPPORT

Found this useful?

BUY ME A COFFEE

The site is free and always will be. If the mock exam helped, you can buy me a coffee. Support goes to keeping the lights on.